

Topic

KernelGen 2.0

Auto Operator Generation & Benchmark Co-development

Jian Tao

Algorithm Engineer, AI Systems Research Group
Beijing Academy of Artificial Intelligence(BAAI)



KennelGen: Exploration and Practice of Automatic Kernel Generation

Tao Jian



CONTENTS

Part 01

Background | Kernel Generation

Part 02

Design | KernelGen

Part 03

Experience | Real-world Usage

Part 04

What's Next



01 Background

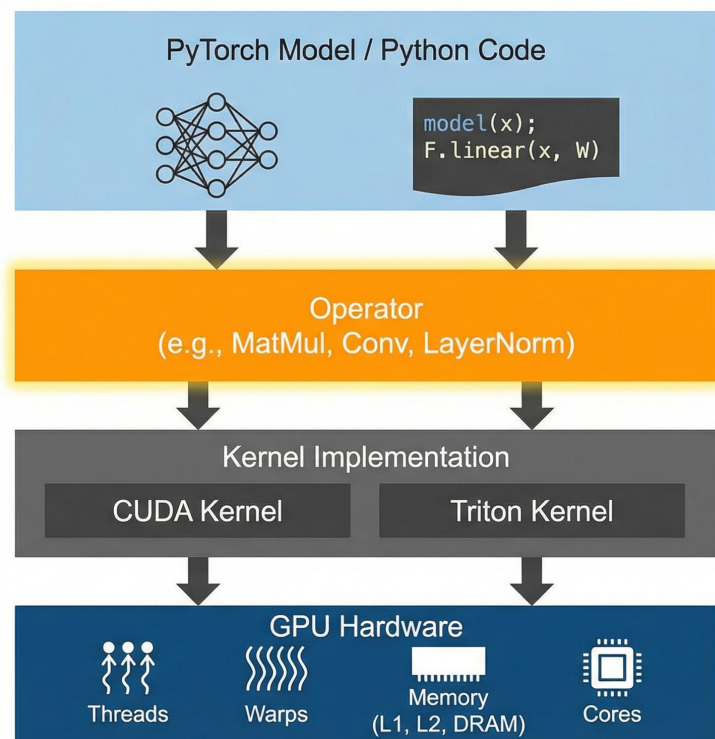


An operator is a computational unit that can be optimized independently. For example, APIs in the PyTorch framework represent operators, and they serve as the building blocks of model training and inference.



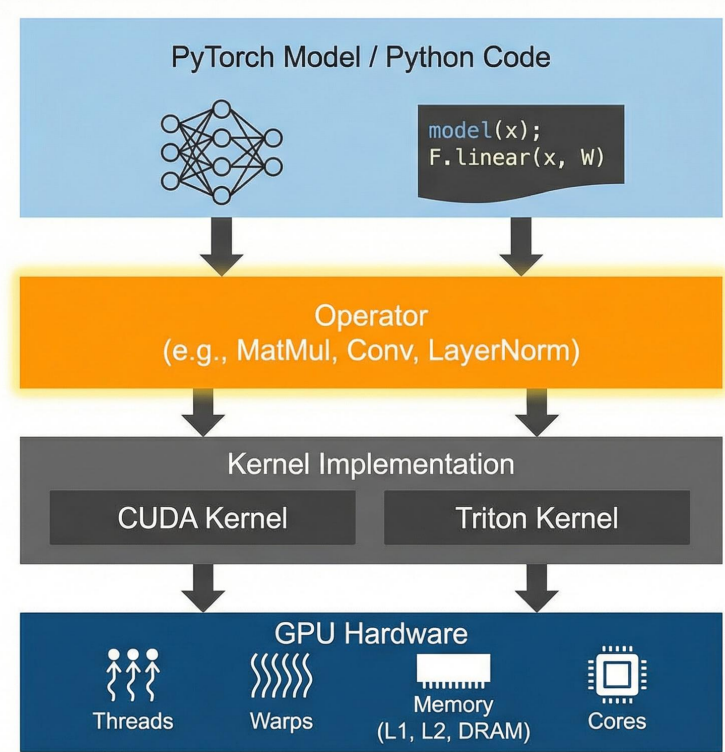
An operator is a computational unit that can be optimized independently. For example, APIs in the PyTorch framework represent operators, and they serve as the building blocks of model training and inference.

PyTorch Operator Call Graph

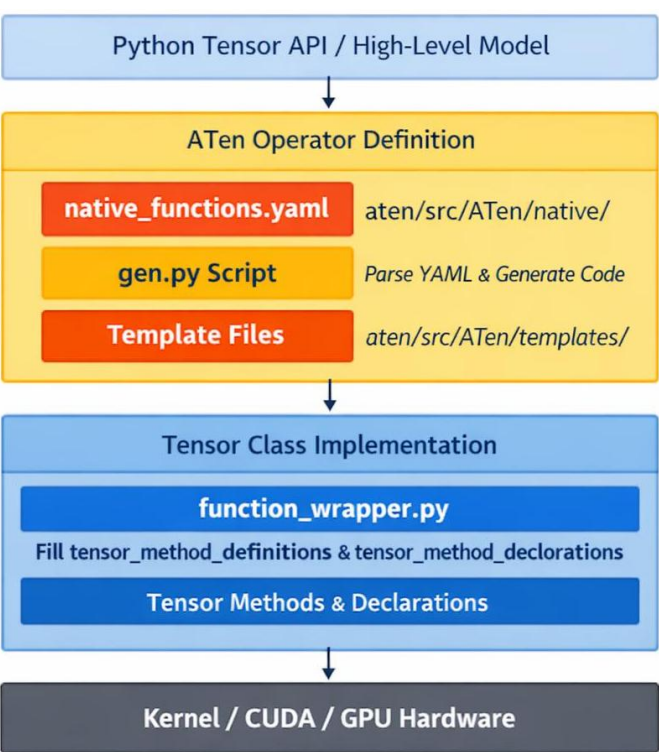


An operator is a computational unit that can be optimized independently. For example, APIs in the PyTorch framework represent operators, and they serve as the building blocks of model training and inference.

PyTorch Operator Call Graph

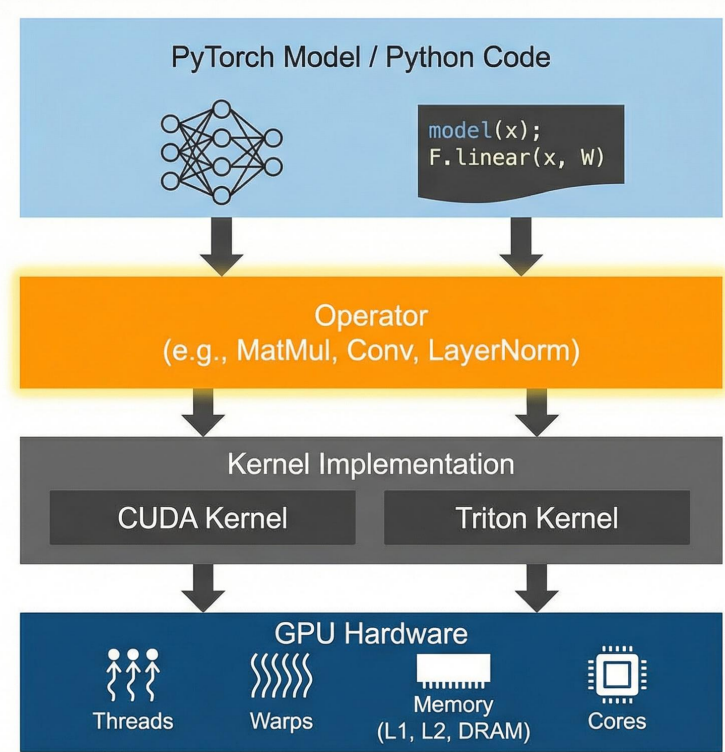


PyTorch ATen Operator Call Details

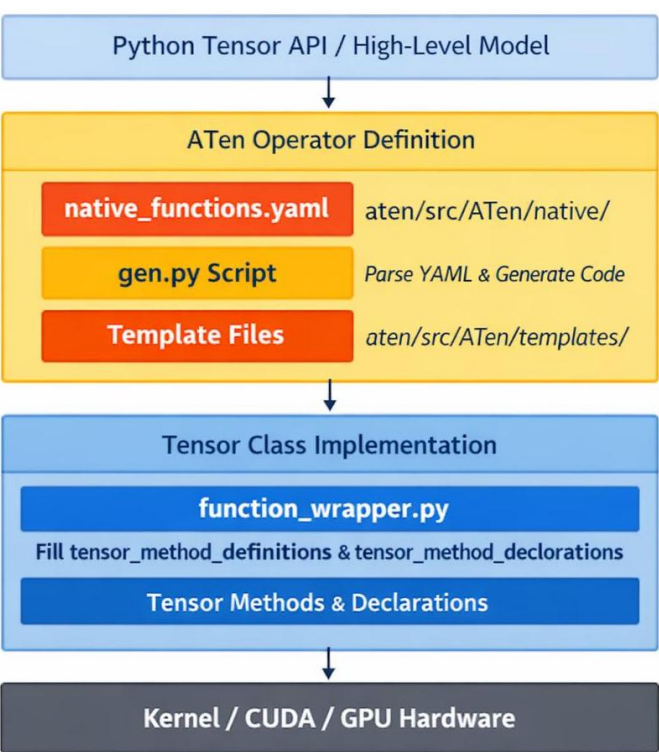


An operator is a computational unit that can be optimized independently. For example, APIs in the PyTorch framework represent operators, and they serve as the building blocks of model training and inference.

PyTorch Operator Call Graph



PyTorch ATen Operator Call Details



Simple CUDA Impl of Addition Operator

```
#include <iostream>
#include <math.h>

// Kernel function to add the elements of two arrays
__global__
void add(int n, float *x, float *y)
{
    for (int i = 0; i < n; i++)
        y[i] = x[i] + y[i];
}

int main(void)
{
    int N = 1<<20;
    float *x, *y;

    // Allocate Unified Memory - accessible from CPU or GPU
    cudaMallocManaged(&x, N*sizeof(float));
    cudaMallocManaged(&y, N*sizeof(float));

    // initialize x and y arrays on the host
    for (int i = 0; i < N; i++) {
        x[i] = 1.0f;
        y[i] = 2.0f;
    }

    // Run kernel on 1M elements on the GPU
    add<<<1, 1>>>(N, x, y);

    // Wait for GPU to finish before accessing on host
    cudaDeviceSynchronize();

    // Check for errors (all values should be 3.0f)
    float maxError = 0.0f;
    for (int i = 0; i < N; i++) {
        maxError = fmax(maxError, fabs(y[i]-3.0f));
    }
    std::cout << "Max error: " << maxError << std::endl;

    // Free memory
    cudaFree(x);
    cudaFree(y);
    return 0;
}
```



Kernel generation bridges the gap between high-level mathematical abstractions and hardware-specific execution, ensuring that operations like matrix multiplication achieve maximum throughput and energy efficiency on a given processor.

This research direction is well-documented in various blogs and papers.



Kernel generation bridges the gap between high-level mathematical abstractions and hardware-specific execution, ensuring that operations like matrix multiplication achieve maximum throughput and energy efficiency on a given processor.

This research direction is well-documented in various blogs and papers.

KernelBench from Stanford

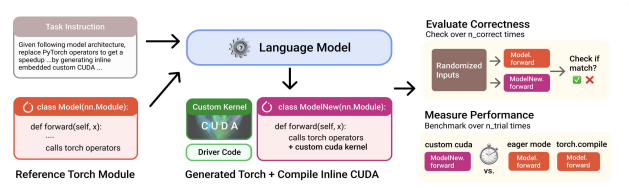


Figure 1: KernelBench evaluates LMs' ability to generate performant GPU Kernels. Overview of tasks in KernelBench: KernelBench tasks LMs with generating optimized CUDA kernels for a given target PyTorch model architecture and conducts automated evaluation

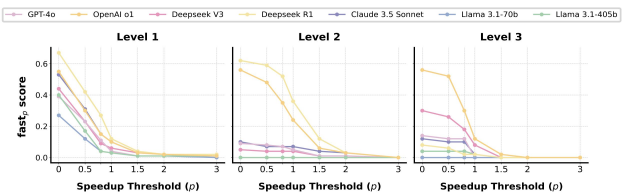


Figure 3: Most LM-generated kernels are slow. This figure shows the distribution of the fast₁ metric as the speedup threshold p (over PyTorch baseline) increases. fast₀ represents the number of correct kernels regardless of speed, and fast₁ represents the number of correct kernels achieving at least $>1\times$ speedup over PyTorch. Increasing the threshold p increases the difficulty.

AutoTriton LLM from Tsinghua

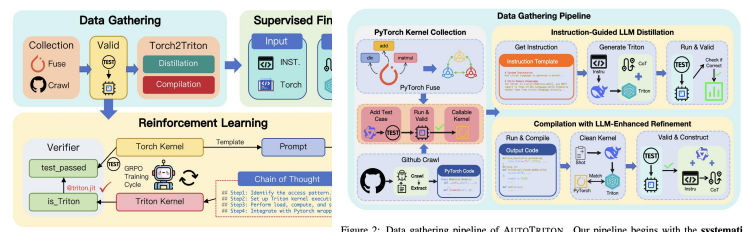


Figure 1: Overview of AUTRITON pipeline. The entire pipeline consists of three collection, SFT stage, and RL stage.

Model	#Params	TRITONBENCH-G			TRITONBENCH-T		
		Call / Exec	fast ₁ / fast ₂		Call / Exec	fast ₁ / fast ₂	
Seed-Coder-Reasoning	8B	2.72 / 2.72	0.54 / 0.00		3.61 / 3.61	1.20 / 0.00	
Qwen3	8B	2.17 / 1.63	0.54 / 0.00		6.02 / 5.42	2.41 / 0.00	
Owen3	32B	10.33 / 9.24	2.17 / 1.63		21.96 / 21.96	10.84 / 3.01	
GPT-4o	-	10.87 / 10.33	4.89 / 1.63		18.67 / 15.06	7.84 / 1.20	
Claude-4-Sonnet	-	9.24 / 9.24	1.64 / 1.09		10.84 / 10.84	4.22 / 1.84	
DeepSeek-R1-0120	671B	13.59 / 13.05	4.89 / 0.54		28.92 / 28.37	22.89 / 3.01	
DeepSeek-R1-0528	685B	16.30 / 15.22	3.80 / 0.54		30.72 / 30.12	11.45 / 4.82	
AUTOTRITON	8B	<u>15.76</u> / 15.76	7.61 / 2.17		40.36 / 39.16	<u>17.04</u> / 6.02	
w/o RL (SFT only)	8B	14.67 / 14.13	4.89 / 1.09		<u>34.94</u> / <u>34.94</u>	15.06 / 7.83	

Table 1: Main results on TRITONBENCH. We present Call Accuracy (Call), Execution Accuracy (Exec), fast₁ and fast₂. The best-performing and second-best-performing methods are highlighted in **Bold** and Underline, respectively.

Workflow from Nvidia

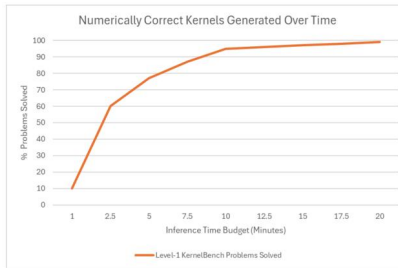
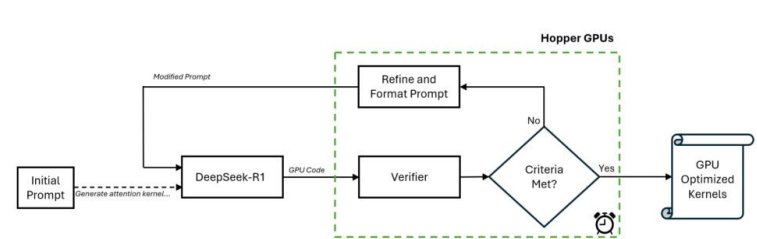
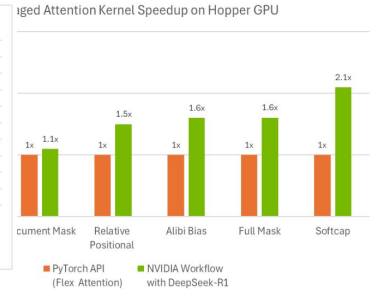


Figure 4: Inference-time scaling results in optimized GPU kernels



02 Design | KernelGen



An automated tool platform for high-performance Triton operators, supporting multiple chip backends. This is a full upgrade of Triton Copilot: expanding from a simple "code assistant" to covering the full operator lifecycle including generation, verification and cross-chip adaptation.

Link: <https://kernelgeen.flagos.io/home>



An automated tool platform for high-performance Triton operators, supporting multiple chip backends. This is a full upgrade of Triton Copilot: expanding from a simple "code assistant" to covering the full operator lifecycle including generation, verification and cross-chip adaptation.

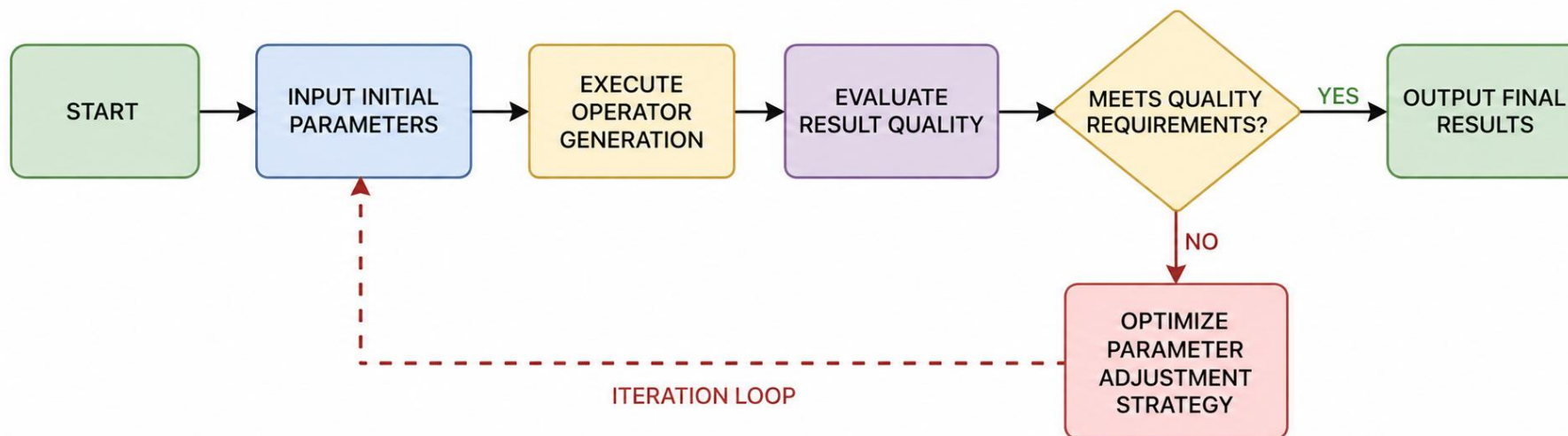
Features

- Natural language descriptions.
- Automated Validation and Testing
- Multi-chip backend verification

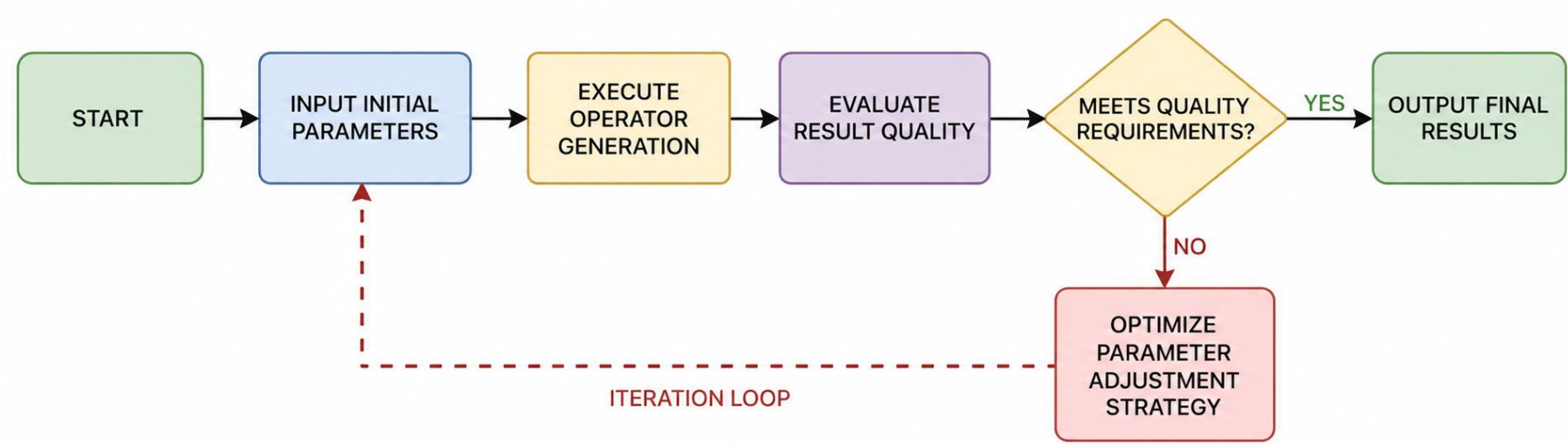
Link: <https://kernelgeen.flagos.io/home>



Overall Pipeline Introduction



Overall Pipeline Introduction



FlagOS | KernelGen

算子生成

算子广场

> 展开查看原始需求

检测到和您需求类似的代码片段，您可以选择一个最佳实现，后续的代码生成将以该片段为参考，我们将及时处理：

☐ https://github.com/harleyszhang/lite_llama/blob/main/tests/kernels/kernels.py

☐ 以上仓库都不合适，直接生成

https://github.com/harleyszhang/lite_llama/blob/main/tests/kernels/kernels.py

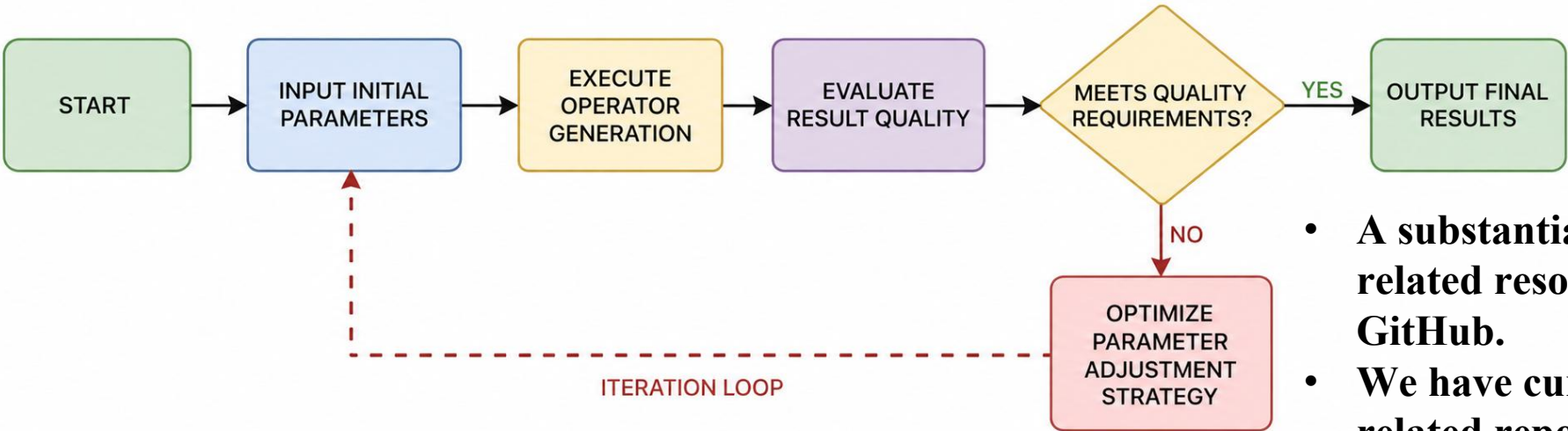
star 163 前往GitHub查看源码 ->

该仓库下，和您需求类似的代码片段如下：

```
1 class RMSNorm(nn.Module):
2     def __init__(self, dim):
3         super(RMSNorm, self).__init__()
4         self.weight = nn.Parameter(torch.ones(dim))
5
6     def forward(self, x):
7         var = torch.mean(x**2, dim=-1, keepdim=True)
8         rms = torch.sqrt(var)
9         return x / rms * self.weight
```



Overall Pipeline Introduction



- A substantial number of Triton-related resources are available on GitHub.
- We have curated **204** Triton-related repositories and **421** CUDA-related repositories to support efficient information retrieval.

FlagOS | KernelGen

算子生成

算子广场

> 展开查看原始需求

检测到和您需求类似的代码片段，您可以选择一个最佳实现，后续的代码生成将以该片段为参考，我们将及时处理：

☐ https://github.com/harleyszhang/lite_llama/blob/main/tests/kernels/kernels.py

☐ 以上仓库都不合适，直接生成

https://github.com/harleyszhang/lite_llama/blob/main/tests/kernels/kernels.py

star 163 前往GitHub查看源码 ->

该仓库下，和您需求类似的代码片段如下：

```
1 class RMSNorm(nn.Module):
2     def __init__(self, dim):
3         super(RMSNorm, self).__init__()
4         self.weight = nn.Parameter(torch.ones(dim))
5
6     def forward(self, x):
7         var = torch.mean(x**2, dim=-1, keepdim=True)
8         rms = torch.sqrt(var)
9         return x / rms * self.weight
```

html_url: "https://github.com/NVlabs/Sana"	star_count: 4717	fork_count: 309	subscribers_count: 76
html_url: "https://github.com/asmjit/asmjit"	star_count: 4334	fork_count: 550	subscribers_count: 144
html_url: "https://github.com/tile-ai/tilelang"	star_count: 3946	fork_count: 318	subscribers_count: 28
html_url: "https://github.com/flash-linear-attention"	star_count: 3861	fork_count: 303	subscribers_count: 30
html_url: "https://github.com/HazyResearch/ThunderKittens"	star_count: 2921	fork_count: 198	subscribers_count: 50
html_url: "https://github.com/pytorch/ao"	star_count: 2507	fork_count: 369	subscribers_count: 44
html_url: "https://github.com/rush/Triton-Puzzles"	star_count: 2118	fork_count: 174	subscribers_count: 10
html_url: "https://github.com/gpu-mode/resource-stream"	star_count: 1792	fork_count: 105	subscribers_count: 46
html_url: "https://github.com/ELS-RD/kernl"	star_count: 1585	fork_count: 97	subscribers_count: 26
html_url: "https://github.com/meta-recsys/generative-recommenders"	star_count: 1548	fork_count: 305	subscribers_count: 26
html_url: "https://github.com/pytorch/FBGEMM"	star_count: 1477	fork_count: 684	subscribers_count: 61
html_url: "https://github.com/ByteDance-Seed/Triton-distributed"	star_count: 1238	fork_count: 105	subscribers_count: 16
html_url: "https://github.com/neuraloperator/neuraloperator"	stars_count: 3048	forks_count: 753	subscribers_count: 38
html_url: "https://github.com/NVIDIA/TransformerEngine"	stars_count: 2890	forks_count: 540	subscribers_count: 36
html_url: "https://github.com/pytorch/TensorRT"	stars_count: 2878	forks_count: 369	subscribers_count: 68

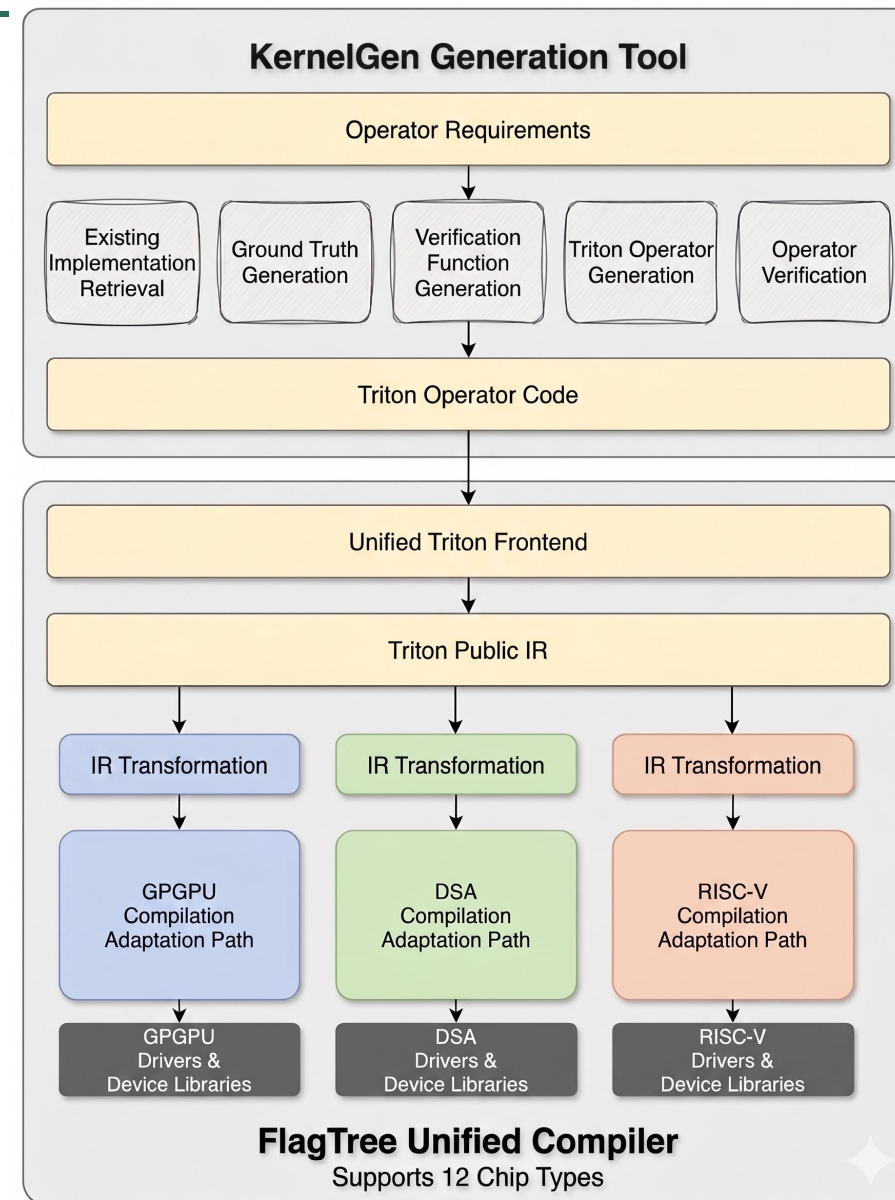


- **Seamless FlagOS Ecosystem: Cross-Platform Triton Operator Support**
 - **Achieve Seamless Support for Generating & Testing & Triton Operators on Multiple Chips**
 - **No Environment Configuration Required**
-
- Huawei Ascend
 - Hygon
 - Moore Threads
 - Iluvatar
 - MetaX



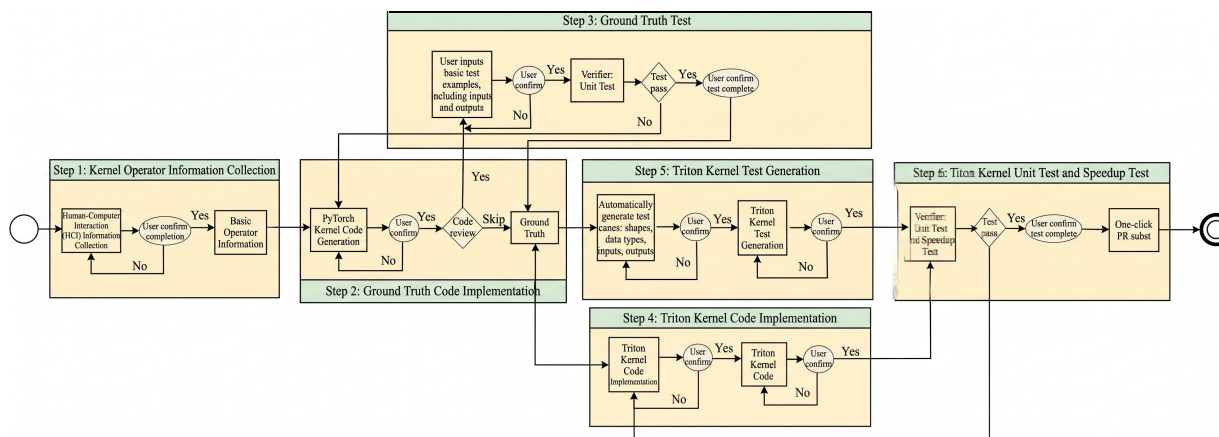
- **Seamless FlagOS Ecosystem: Cross-Platform Triton Operator Support**
- **Achieve Seamless Support for Generating & Testing & Triton Operators on Multiple Chips**
- **No Environment Configuration Required**

- Huawei Ascend
- Hygon
- Moore Threads
- Iluvatar
- MetaX



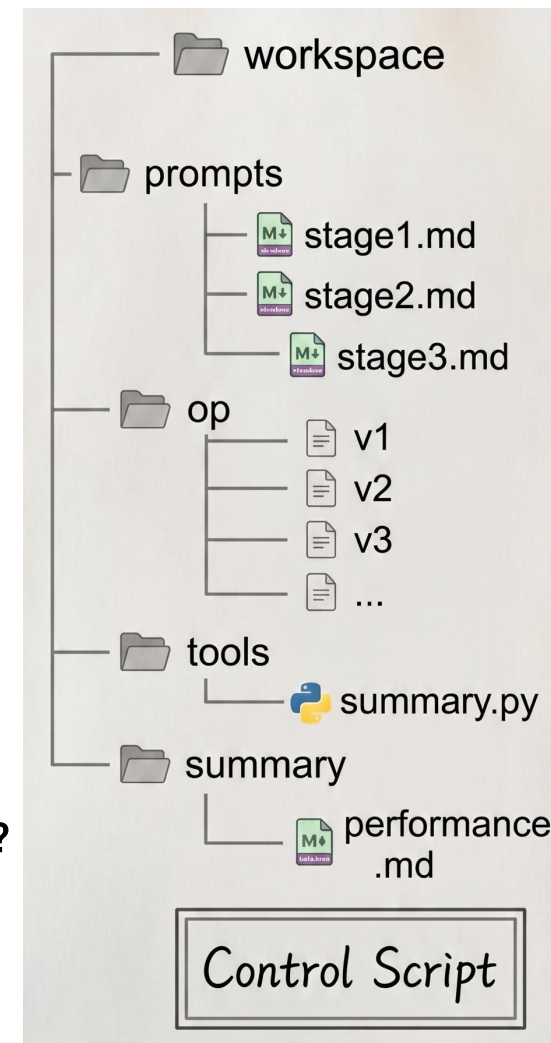
- KernelGen v1.0

- LLM + workflow
- Drawing on the expertise of kernel development specialists, we designed a customized workflow and implemented the entire pipeline using a LangChain-inspired architecture.



- KernelGen v2.0

- Given that we have access to agents like **Claude Code** and **Gemini CLI**, shouldn't we just leverage them for this specific task instead of building something custom?
- When we use these agents for **long tasks**, it's easy to **lose focus** or **stray** from the



Comprehensively enhance the generation capabilities of various AI chip kernels

Features

- The chip support has been expanded to **six** vendors
- Supports Triton & **Trion-TLE**
- Supported interfaces: VS Code, Claude Code, OpenClaw, etc.
- Products: web, skills, MCP server

link: <https://kernelgen.flagos.io/home>
gitub: <https://github.com/flagos-ai/kernelgen>

The screenshot shows the KernelGen website with a navigation bar including 'FlagOS | KernelGen', 'Cases', 'Products', 'Features', 'Ecosystem', 'Community', 'GitHub', and a 'Login/Sign up' button. The main heading is 'Build Kernels in Minutes with Multi-Backend Support'. Below it, a subheading states: 'KernelGen is an AI-powered Triton Kernel development tool, supporting operator generation, optimization, specialization, multi-backend, multi-framework integration, and minute-level delivery.' There are two buttons: 'Start Building for Free' and 'Watch Demo'. A section below highlights three key benefits: '10x Development Efficiency Boost', '6+ Backend Support', and '>CUDA Performance Optimization'. At the bottom, logos for six vendors are displayed: Ascend, 天微电子 (Tianwei Electronics), HYGON, METAX, 摩尔线程 (Moore Threads), and NVIDIA.

FlagOS | KernelGen Cases Products Features Ecosystem Community GitHub Login/Sign up

AI-Powered Kernel Development

Build Kernels in Minutes with Multi-Backend Support

KernelGen is an AI-powered Triton Kernel development tool, supporting operator generation, optimization, specialization, multi-backend, multi-framework integration, and minute-level delivery.

Start Building for Free Watch Demo

10x Development Efficiency Boost

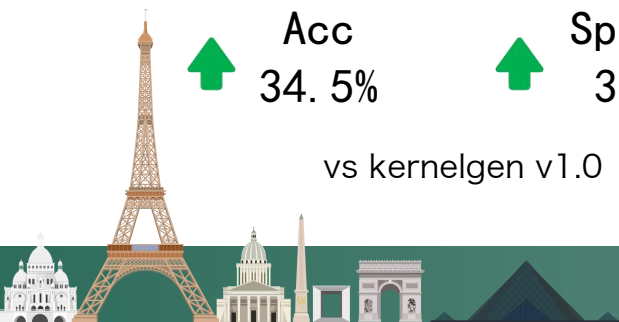
6+ Backend Support

>CUDA Performance Optimization

Ascend 天微电子 Huavate CoreX HYGON 中科海光 METAX 摩尔线程 MOORE THREADS NVIDIA

↑ Acc 34.5% ↑ Speedup 3.59x

vs kernelgen v1.0



03

Experience | Real-world Usage



- KernelGenBench
 - 110 aten operators
 - 50 cublas operators
 - 50 vllm operators
 - More test cases for operators

```
@parametrize("M, N, K", MNK_SHAPES)
@parametrize("scalar", SCALARS)
@parametrize("dtype", FLOAT_DTYPES)
def test_accuracy_baddbmm_backward(M, N, K, scalar, dtype):
    batch = 2
    mat1 = torch.randn((batch, M, K), dtype=dtype, device=device, requires_grad=True)
    mat2 = torch.randn((batch, K, N), dtype=dtype, device=device, requires_grad=True)
    bias = torch.randn((batch, M, N), dtype=dtype, device=device, requires_grad=True)
```

- Shapes from real-world LLM inference
- Supports evaluation of multiple non-Nvidia chips

	Time ↓	Tokens ↓	Acc ↑	Speedup ↑
KernelGen 2.0	14m10s	10K	100%	1.14x
Claude Code	34m42s	21.6K	100%	0.64x

Table 2: Cross-platform evaluation on 110 ATen operators across six chips (NVIDIA, Ascend, MUSA, Hygon, MetaX, Iluvatar), showing how correctness and speedup transfer—or fail to—across heterogeneous hardware backends.

Method & Setup	NVIDIA (CUDA)		Ascend (CANN)		MUSA (MUSIFY)		Hygon (ROCm)		MetaX (MACA)		Iluvatar (CoreX)	
	Acc (%)	Spd (x)	Acc (%)	Spd (x)	Acc (%)	Spd (x)	Acc (%)	Spd (x)	Acc (%)	Spd (x)	Acc (%)	Spd (x)
LLM Sampling Methods												
Pass@1 (Opus-4.6)	39	0.90	46	0.19	44	0.69	37	0.98	38	0.89	38	0.88
Pass@1 (MiniMax M-2.5)	4	0.88	4	0.25	6	1.33	4	1.05	5	0.77	4	1.15
Pass@1 (Qwen3.5-27b)	8	0.83	9	0.09	3	1.02	7	0.90	7	0.98	10	1.03
Pass@5 (Opus-4.6)	62	0.79	63	0.15	60	0.74	54	0.92	65	0.68	57	0.83
Pass@5 (MiniMax M-2.5)	21	0.76	17	0.20	15	0.53	12	1.05	8	0.33	9	0.76
Pass@5 (Qwen3.5-27b)	13	1.04	16	0.18	11	1.10	15	0.72	10	0.99	17	1.02
Vanilla Agentic Frameworks												
Claude Code (Opus-4.6)	92	0.86	89	0.18	93	0.80	88	0.87	96	0.89	83	0.83
Claude Code (GLM-5.0)	67	0.83	65	0.16	-	-	65	0.81	59	0.90	-	-
Claude Code (MiniMax M-2.5)	69	0.78	69	0.16	74	0.59	73	0.72	83	0.63	69	0.58
Claude Code (Qwen3.5-27b)	80	0.68	78	0.25	75	0.61	75	0.85	-	-	-	-
Kernel-Specialized Agents												
AKO4all (Opus-4.6)	89	1.00	84	0.30	88	1.09	88	1.08	86	1.12	90	1.07
CUDA Opt. Skill (MiniMax M-2.5)	63	0.81	53	0.21	64	0.77	65	0.81	67	0.77	58	0.79
AutoKernel (GLM-5.0)	87	1.00	53	0.82	56	1.01	64	0.99	59	1.00	-	-
AutoKernel (MiniMax M-2.5)	-	-	61	0.66	71	1.36	66	0.99	71	1.04	50	1.02
AutoKernel (Qwen3.5-27b)	-	-	40	0.37	75	1.03	65	1.00	74	1.04	-	-



- We have contributed almost 200 pytorch operators to FlagGems, which is the largest triton operator repo under FlagOS.
OpenSource

FlagGems

40

high-quality

Experimental

165

Filters

is:pr is:open kernelgen

Clear current search query, filters, and sorts

72 Open

✓

119 Closed

☐	[KernelGen] Add optimized repeat operator with 1.37x speedup × KernelGen size/Large vendor/lluvatar	
#2174 opened 2 days ago by zacliu2023 · Review required 5 tasks done		
☐	[KernelGen] Add randn_like and randn on Ascend ✓ KernelGen size/Medium vendor/Ascend	2
#2173 opened 2 days ago by ftgreat 3 tasks		
☐	[KernelGen] Fix gather and add scatter on Ascend ✓ KernelGen size/Large vendor/Ascend	4
#2172 opened 2 days ago by ftgreat 1 of 3 tasks		
☐	[KernelGen] Add zeros_like and zeros on Ascend ✓ KernelGen size/Small vendor/Ascend	4
#2171 opened 2 days ago by ftgreat 3 tasks		
☐	[KernelGen] Add view_as_complex operator × KernelGen size/Large vendor/lluvatar	
#2169 opened 2 days ago by zacliu2023 · Review required 4 tasks done		
☐	[KernelGen] Add optimized ones_like operator with 1.15x speedup × KernelGen size/Large vendor/lluvatar	
#2168 opened 2 days ago by zacliu2023 · Review required 5 tasks done		
☐	[KernelGen] Add optimized add operator with 1.01x speedup × KernelGen size/Large vendor/lluvatar	3
#2167 opened 2 days ago by zacliu2023 · Review required 5 tasks done		
☐	[KernelGen] Add optimized clamp operator with 1.0x speedup × KernelGen size/Large vendor/lluvatar	
#2166 opened 3 days ago by zacliu2023 · Review required 4 tasks done		
☐	[KernelGen] Add optimized zeros operator with 1.16x speedup × KernelGen size/Medium vendor/lluvatar	
#2165 opened 3 days ago by zacliu2023 · Review required 5 tasks done		
☐	[KernelGen] Add exponential_operator for lluvatar platform × KernelGen size/Medium vendor/lluvatar	
#2164 opened 3 days ago by zacliu2023 · Review required 6 tasks done		
☐	[KernelGen] Add optimized resolve_conj operator with 1.68x speedup × KernelGen size/Small vendor/lluvatar	
#2163 opened 3 days ago by zacliu2023 · Review required 5 tasks done		
☐	[KernelGen] Add optimized embedding operator with 13.10x speedup × KernelGen size/Medium vendor/lluvatar	
#2161 opened 3 days ago by zacliu2023 · Review required 7 tasks done		
☐	[KernelGen] Add optimized polar operator with 1.85x speedup × KernelGen size/Medium vendor/lluvatar	
#2159 opened 3 days ago by zacliu2023 · Review required 7 tasks done		



- During the adaptation of the **DeepSeek v4** model with **FlagOS**, we extensively adopted **KernelGen** for operator **generation** and **optimization**.
- Reference implementation is **Tilelang** from deepseek v4.

Operator	Functionality	Comparison with TileLang	Development Method
Act Quant	Activation Quantization (fast_round_scale, bitwise implementation of log2_ceil + pow2)	1.42x uplift	Expert
hc_split_sinkhorn	Multi-head Latent Attention Preprocessing (GEMM + Post-processing fusion into a single kernel)	2.73x uplift	Expert
FP8 MatMul	FP8 Matrix Multiplication (128x128 block-wise scaling, support for fp8e8m0 scale)	2.47x uplift	KernelGen
Sparse Attention	Sparse Attention (attention-sink + top-k selection, using tl.dot as a substitute for vector dot)	1.89x uplift	KernelGen
Hadamard Transform	Fast Hadamard Transform (butterfly stages, support for non-power-of-2 dimensions)	1.13x uplift	KernelGen



- Adaptation pipeline:

- Setup

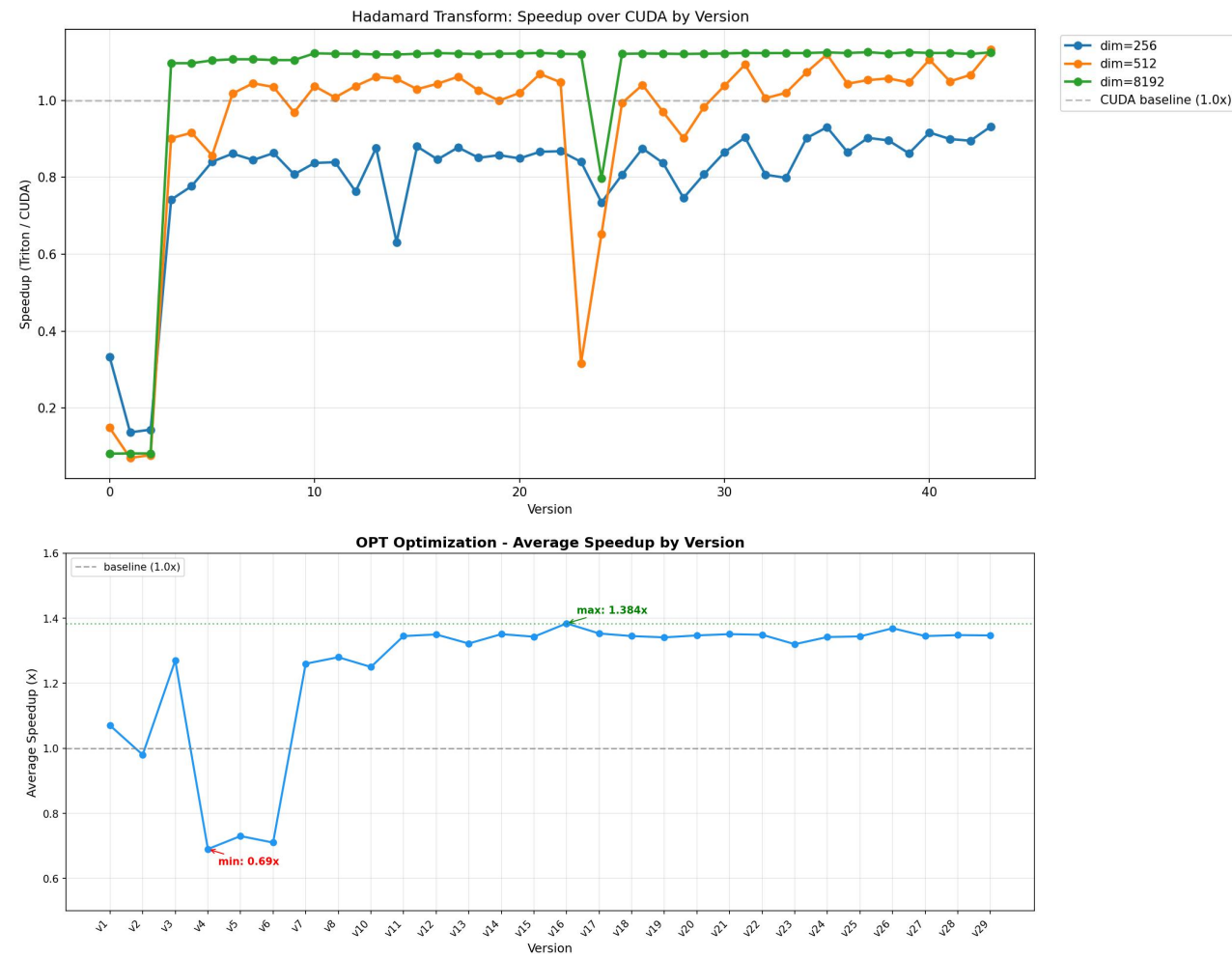
- Test file
 - Benchmark file
 - Reference implementation

- Generation

- Generate a baseline implementation which could pass the accuracy test

- Optimize

- Use KernelGen to optimize the kernel automatically

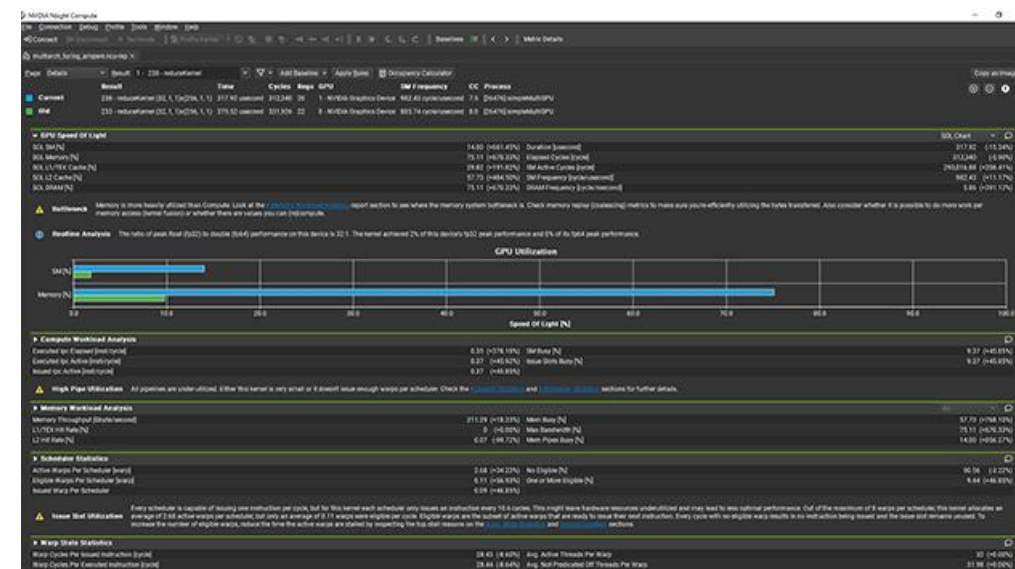
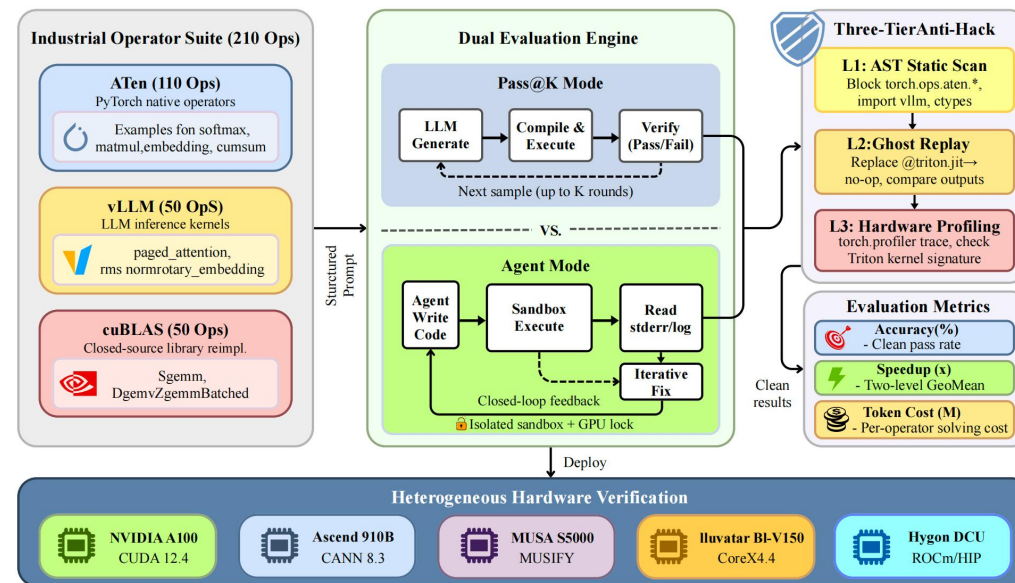


04

What's Next



- KernelGenBench for multichip support
 - For now KernelGenBench only supports multichip for Aten-level operators, but not vllm and cublas kernels
- Add kernel-level profiling tools for multi-chip like NCU(Nvidia Nsight Compute) in Nvidia devices
- Self-evolved Agents for KernelGen via ever-expanding knowledge base



Thanks!

